```
# run this code as first notebook cell to render plotly graphs in pdf
import plotly.io as pio
pio.renderers.default = "notebook"

import plotly.graph_objects as go
# Monkey-patch to make ALL plots responsive with auto-sized width
original_fig_init = go.Figure.__init__
def new_init(self, *args, **kwargs):
    original_fig_init(self, *args, **kwargs)
    self.update_layout(
        autosize=True,
        width=None,   # remove fixed width
        height=None,  # remove fixed height
        margin=dict(l=10, r=10, t=30, b=10)
    )
go.Figure.__init__ = new_init
```

Basic Plotly Charts

What I'm Doing

In this lab, I **explore Plotly's two main interfaces** - `plotly.graph_objects` (**low-level**) and `plotly.express` (**high-level**) - to **build seven different chart types**. I first practice each chart on synthetic data, then apply the same techniques to a real-world airline on-time performance dataset to surface patterns in flight distances, delays, airline traffic, and destination distributions.

1. Setting Up

I start by importing the four libraries I'll use throughout the lab.

```
#%pip install nbformat

import pandas as pd
import numpy as np
import plotly.graph_objects as go
import plotly.express as px
```

2. Exploring Chart Types with Plotly Graph Objects

The `plotly.graph_objects` module gives me fine-grained control. The pattern is the **same every time**: create an empty `go.Figure()`, add traces with `.add_trace()`, then update the layout with `.update_layout()`.

2.1 **Scatter Plot** - Income vs Age

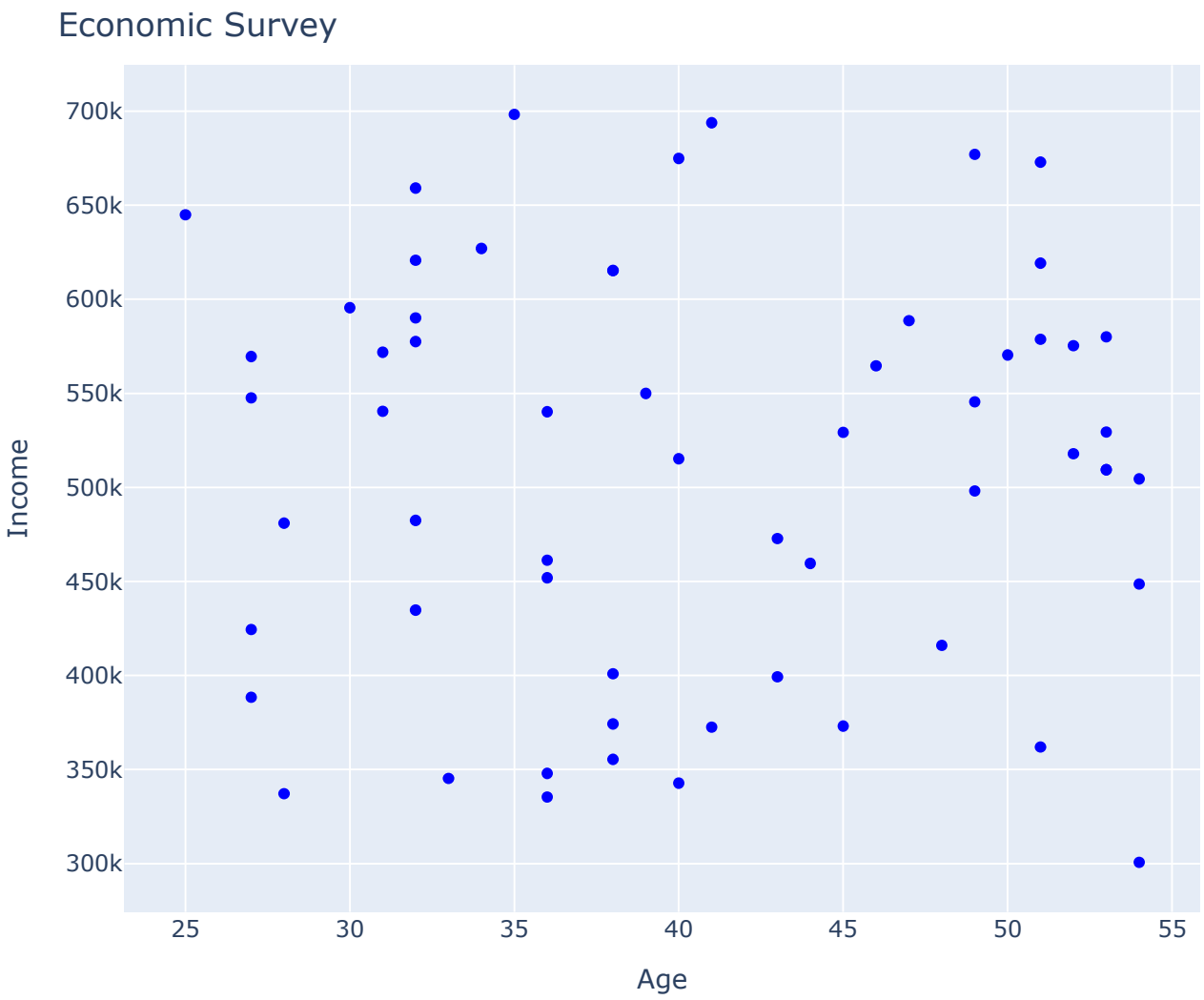
A scatter plot **shows the relationship between two numeric variables**. I generate random age and

income arrays and plot them.

```
age_array = np.random.randint(25, 55, 60)


# generate 3mil data points ranging from 300k-700k
income_array = np.random.randint(300000, 700000, 3000000)

fig = go.Figure()
fig.add_trace(go.Scatter(x=age_array, y=income_array, mode='markers',
marker=dict(color='blue')))
fig.update_layout(title='Economic Survey', xaxis_title='Age',
yaxis_title='Income')
fig.show()
```



`Result:` 3 million incomes distributed across 60 age categories, repeated cyclically. **No clear correlation** emerges - showing that income does not consistently increase or decrease with age in this random sample.

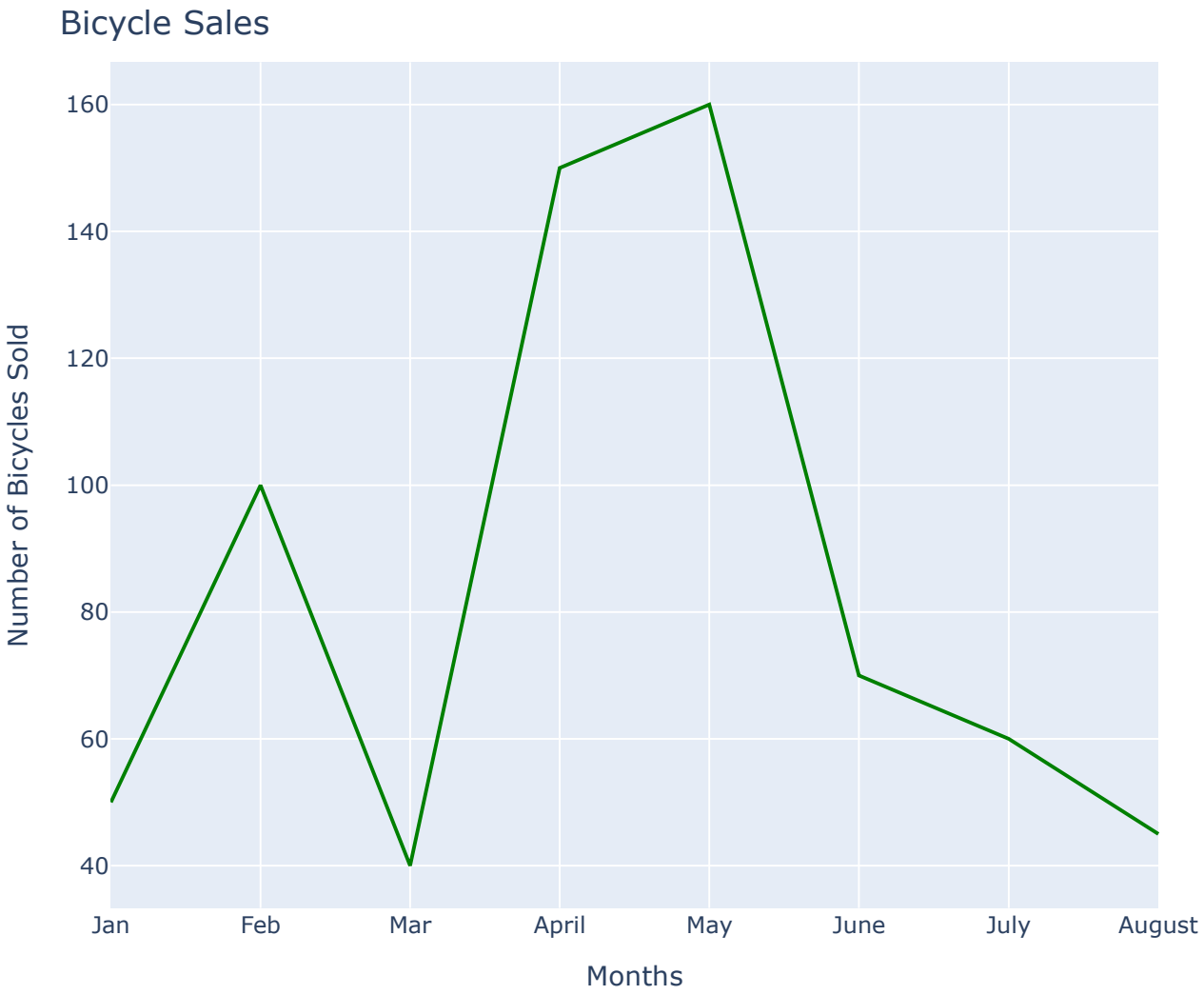
2.2 **Line Plot** - Bicycle Sales Over Time

A line plot connects data points to **show change over time**. I plot monthly bicycle sales across eight months.

```
numberofbicyclessold_array = [50, 100, 40, 150, 160, 70, 60, 45]
```

```
months_array = ["Jan", "Feb", "Mar", "April", "May", "June", "July", "August"]

fig = go.Figure()
fig.add_trace(go.Scatter(x=months_array, y=numberofbicyclessold_array,
mode='lines', marker=dict(color='green')))
fig.update_layout(title='Bicycle Sales', xaxis_title='Months',
yaxis_title='Number of Bicycles Sold')
fig.show()
```



`Result:` Sales peak in May at 160 units, then decline steadily through the summer - a clear **seasonal pattern**.

3. Exploring Chart Types with Plotly Express

`plotly.express` is a high-level wrapper that lets me create a complete figure in a **single function call**, using a much simpler syntax.

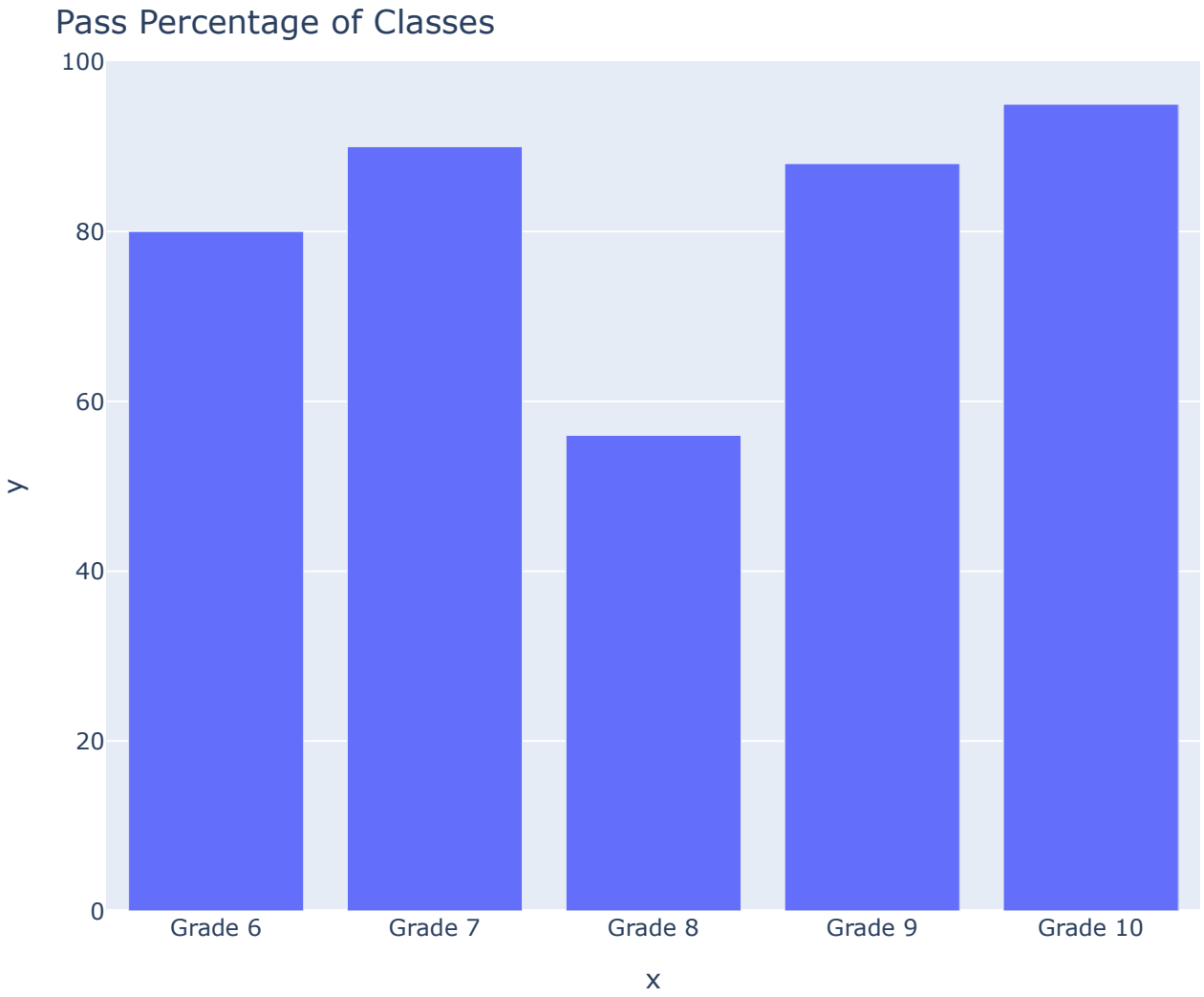
3.1 Bar Plot - Pass Percentage by Grade

A bar chart **compares values across categories**. I plot average pass percentages from Grade 6 to Grade 10.

```
pass_rate_array = [80, 90, 56, 88, 95]
```

```
grade_array = ['Grade 6', 'Grade 7', 'Grade 8', 'Grade 9', 'Grade 10']

fig = px.bar(x=grade_array, y=pass_rate_array, title='Pass Percentage of
Classes')
fig.show()
```



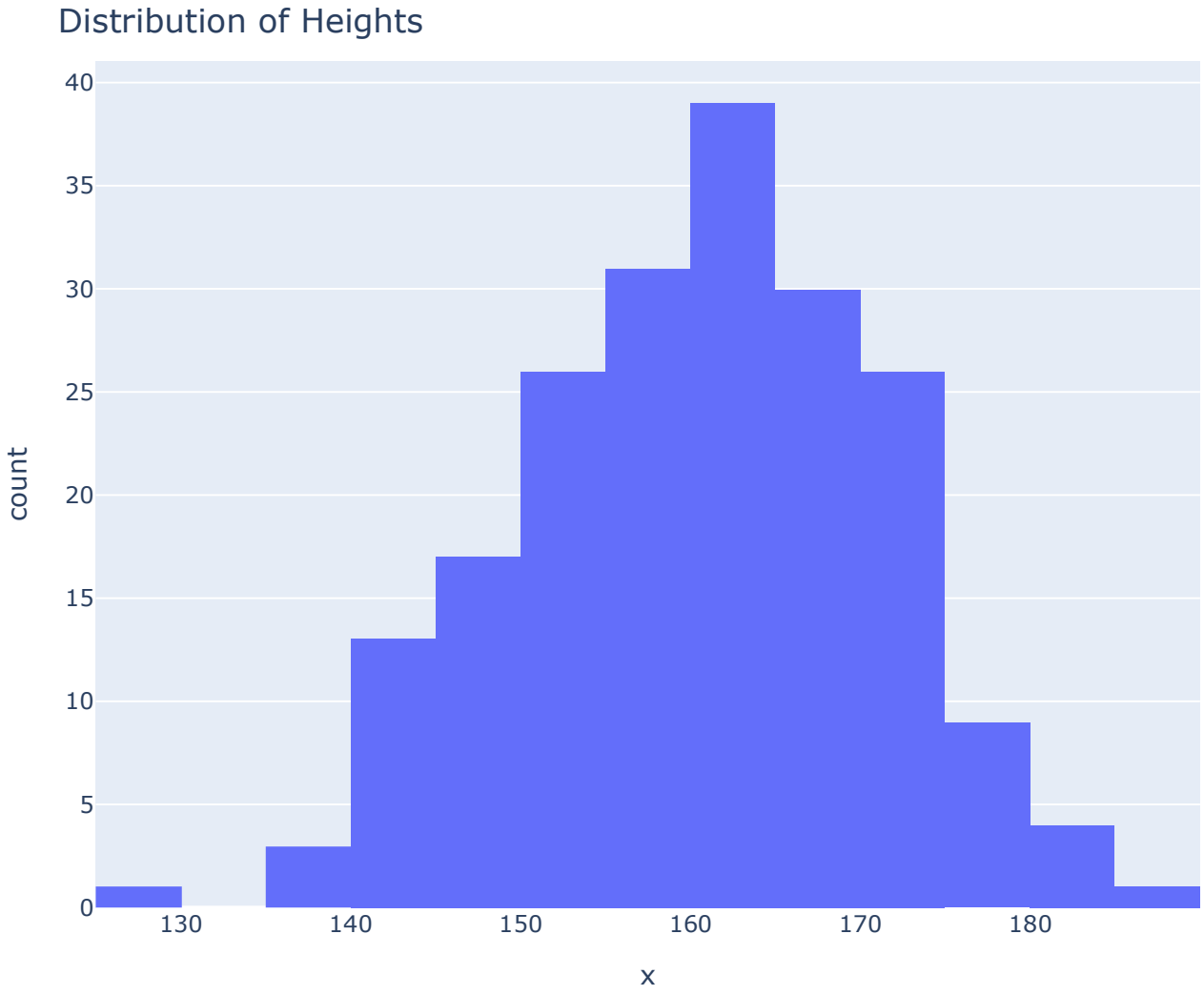
Result: Grade 8 has the lowest pass percentage (56%), while Grade 10 leads at 95%.

3.2 **Histogram** - Distribution of Heights

A histogram shows the **frequency distribution of a continuous variable**. I generate 200 normally distributed heights centered at 160 cm.

```
heights_array = np.random.normal(160, 11, 200)

fig = px.histogram(x=heights_array, title="Distribution of Heights")
fig.show()
```



Result: The tallest bar sits around 160 cm; frequencies taper off toward both tails.

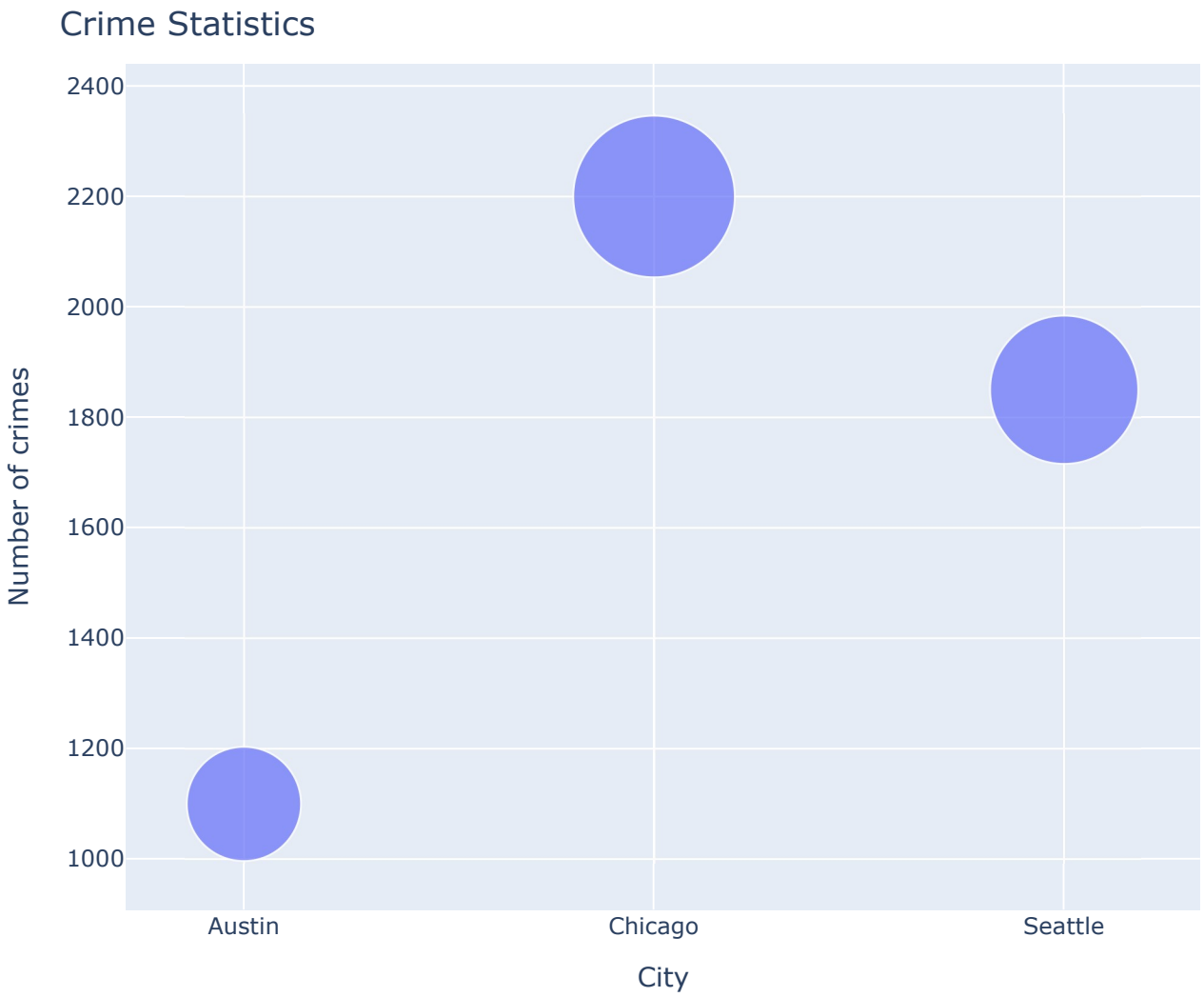
3.3 **Bubble Plot** - Crime Statistics by City

A bubble plot **extends a scatter plot by encoding a third variable in the marker size**. I aggregate crime data by city across two years.

```
crime_details = {
    'City': ['Chicago', 'Chicago', 'Austin', 'Austin', 'Seattle', 'Seattle'],
    'Number of crimes': [1000, 1200, 400, 700, 350, 1500],
    'Year': ['2007', '2008', '2007', '2008', '2007', '2008'],
}
df = pd.DataFrame(crime_details)

## Group the number of crimes by city and find the total number of crimes per
city
bub_data = df.groupby('City')['Number of crimes'].sum().reset_index()

fig = px.scatter(bub_data, x="City", y="Number of crimes", size="Number of
crimes",
                 hover_name="City", title='Crime Statistics', size_max=60)
fig.show()
```



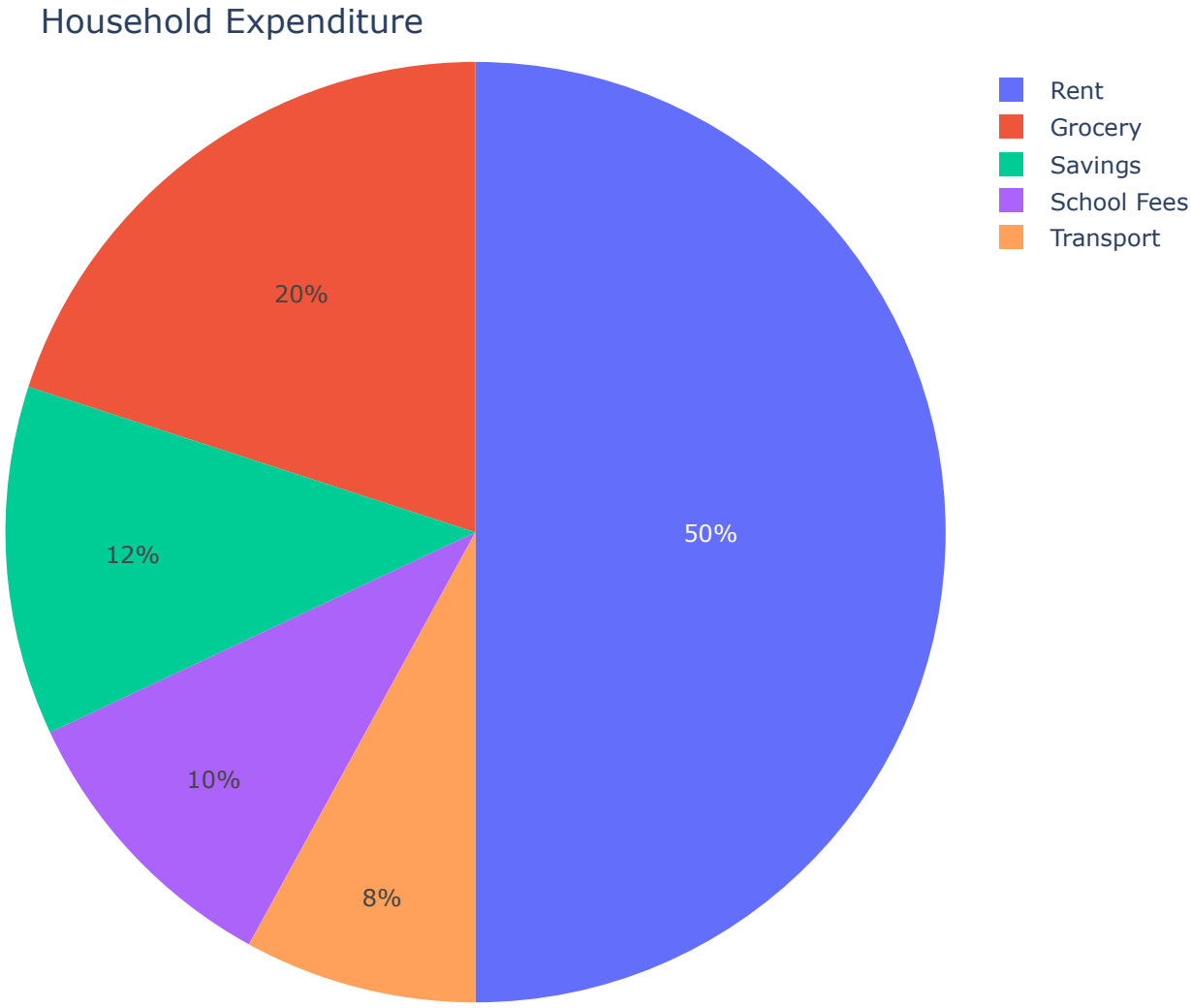
`Result:` Chicago shows the largest bubble, indicating the highest total crime count among the three cities.

3.4 **Pie Plot** - Household Expenditure

A pie chart shows how **a whole is divided into proportional slices.**

```
exp_percent = [20, 50, 10, 8, 12]
house_holdcategories = ['Grocery', 'Rent', 'School Fees', 'Transport', 'Savings']

fig = px.pie(values=exp_percent, names=house_holdcategories, title='Household Expenditure')
fig.show()
```



Result: Rent consumes half the budget (50%), by far the largest slice.

3.5 **Sunburst Chart** - Family Hierarchy

A sunburst chart reveals **hierarchical structure as concentric rings**, with the root at the center.

```
data = dict(
    character=["Eve", "Cain", "Seth", "Enos", "Noam", "Abel", "Awan", "Enoch",
"Azura"],
    parent=["", "Eve", "Eve", "Seth", "Seth", "Eve", "Eve", "Awan", "Eve"],
    value=[10, 14, 12, 10, 2, 6, 6, 4, 4])

fig = px.sunburst(data, names='character', parents='parent', values='value',
title="Family Chart")
fig.show()
```

Family chart

Result: Eve is the **root in the innermost ring**; her **children** (Cain, Seth, Abel, Awan, Azura) occupy the **middle** ring; **grandchildren** like Enoch appear in the **outermost** ring - the hierarchy reads clearly from center outward.

4. Loading and Preparing the Airline Dataset

I now move to the **Reporting Carrier On-Time Performance Dataset**, which tracks approximately 200 million domestic US flights reported to the Bureau of Transportation Statistics. Preview data, dataset metadata, and data glossary here.

```
airline_data = pd.read_csv(
    'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/airline_data.csv',
    encoding="ISO-8859-1",
    dtype={'Div1Airport': str, 'Div1TailNum': str, 'Div2Airport': str,
'Div2TailNum': str}
)
airline_data.head()
```

	Unnamed: 0	Year	Quarter	Month	DayofMonth	DayOfWeek	FlightDate	Reporting_Airline	DO
0	1295781	1998	2	4	2	4	1998-04-02	AS	
1	1125375	2013	2	5	13	1	2013-05-13	EV	
2	118824	1993	3	9	25	6	1993-09-25	UA	
3	634825	1994	4	11	12	6	1994-11-12	HP	
4	1888125	2017	3	8	17	4	2017-08-17	UA	

5 rows × 110 columns

```
airline_data.shape
```

```
(27000, 110)
```

Result: The full dataset spans 270000 of rows across 110 columns.

Since the full dataset is large, I take a random sample of **500 rows with a fixed random seed** so my results are **reproducible**.

```
data = airline_data.sample(n=500, random_state=42)
data.shape
```

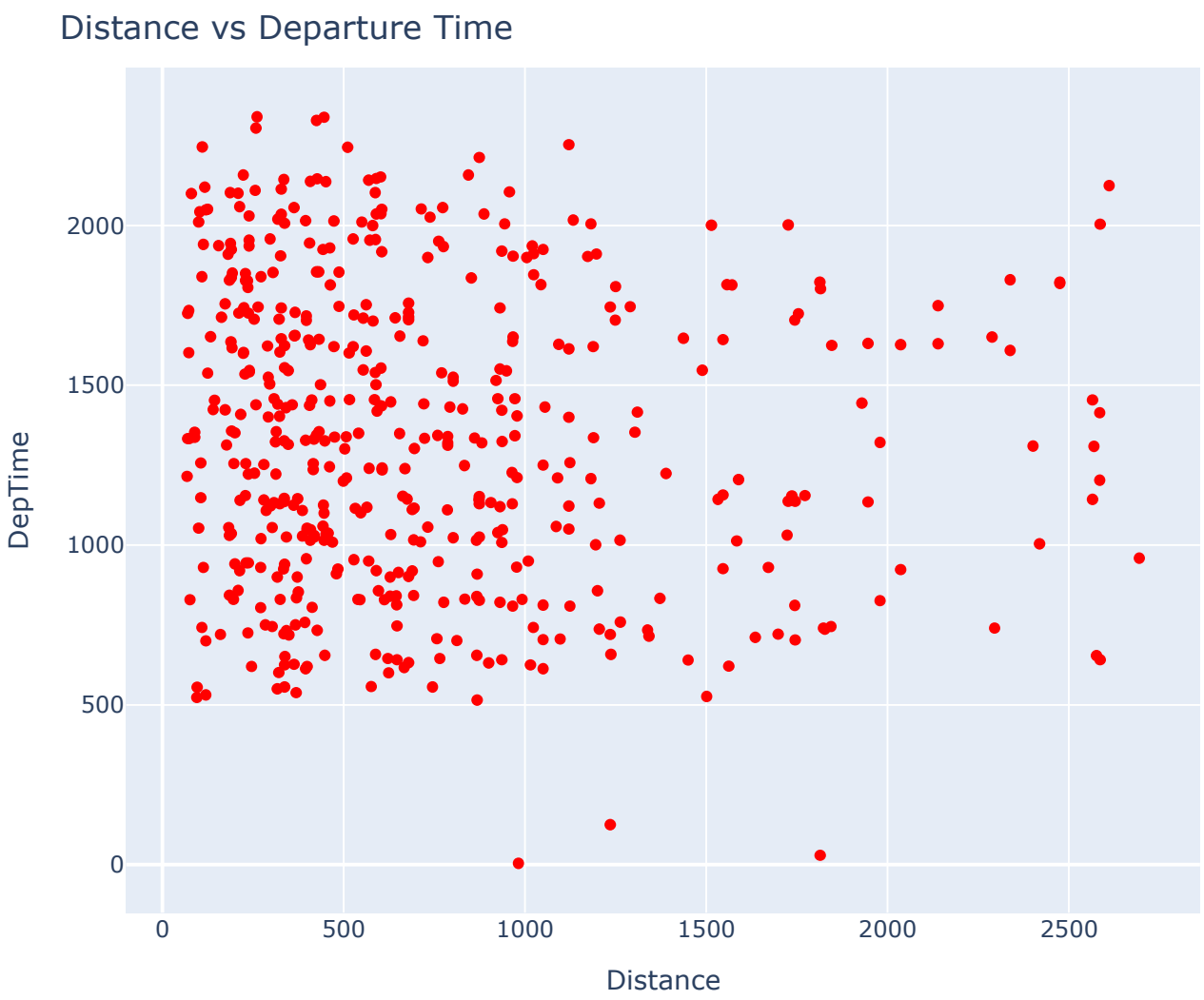
```
(500, 110)
```

This trimmed dataset will power the tasks that follow.

4.1. **Scatter Plot** - Distance vs Departure Time

I want to see how departure times are distributed across different flight distances.

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=data['Distance'], y=data['DepTime'], mode='markers',
marker=dict(color='red')))
fig.update_layout(title='Distance vs Departure Time', xaxis_title='Distance',
yaxis_title='DepTime')
fig.show()
```



`Result:` Shorter distances see departures throughout the entire day, while long-distance flights are more clustered at specific times (limited flights through the day) - likely reflecting scheduled long-haul operations rather than frequent short-hop service.

4.2. **Line Plot** - Month vs Average Flight Delay

I want to identify which months tend to have the worst arrival delays.

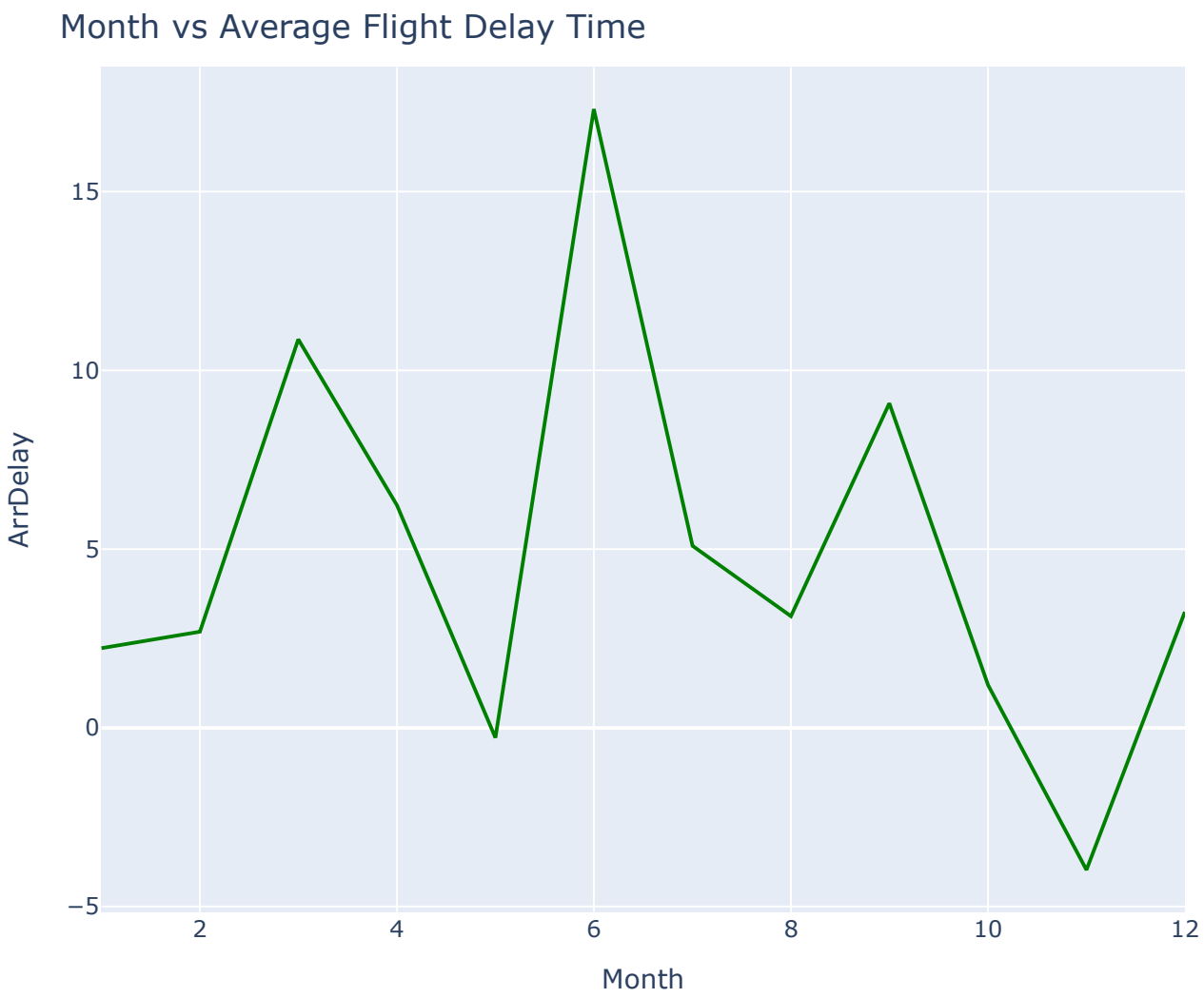
First, I group the data by month and compute the mean arrival delay for each.

```
line_data = data.groupby('Month')['ArrDelay'].mean().reset_index()
line_data.head()
```

	Month	ArrDelay
0	1	2.232558
1	2	2.687500
2	3	10.868421
3	4	6.229167
4	5	-0.279070

Result: A two-column table with each month (1–12) and its corresponding average arrival delay in minutes.

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=line_data['Month'], y=line_data['ArrDelay'],
mode='lines', marker=dict(color='green')))
fig.update_layout(title='Month vs Average Flight Delay Time',
xaxis_title='Month', yaxis_title='ArrDelay')
fig.show()
```



Result: June has the highest average arrival delay, pointing to **summer travel congestion** as a major contributor to late arrivals.

4.3. **Bar Chart** - Flights by Destination State

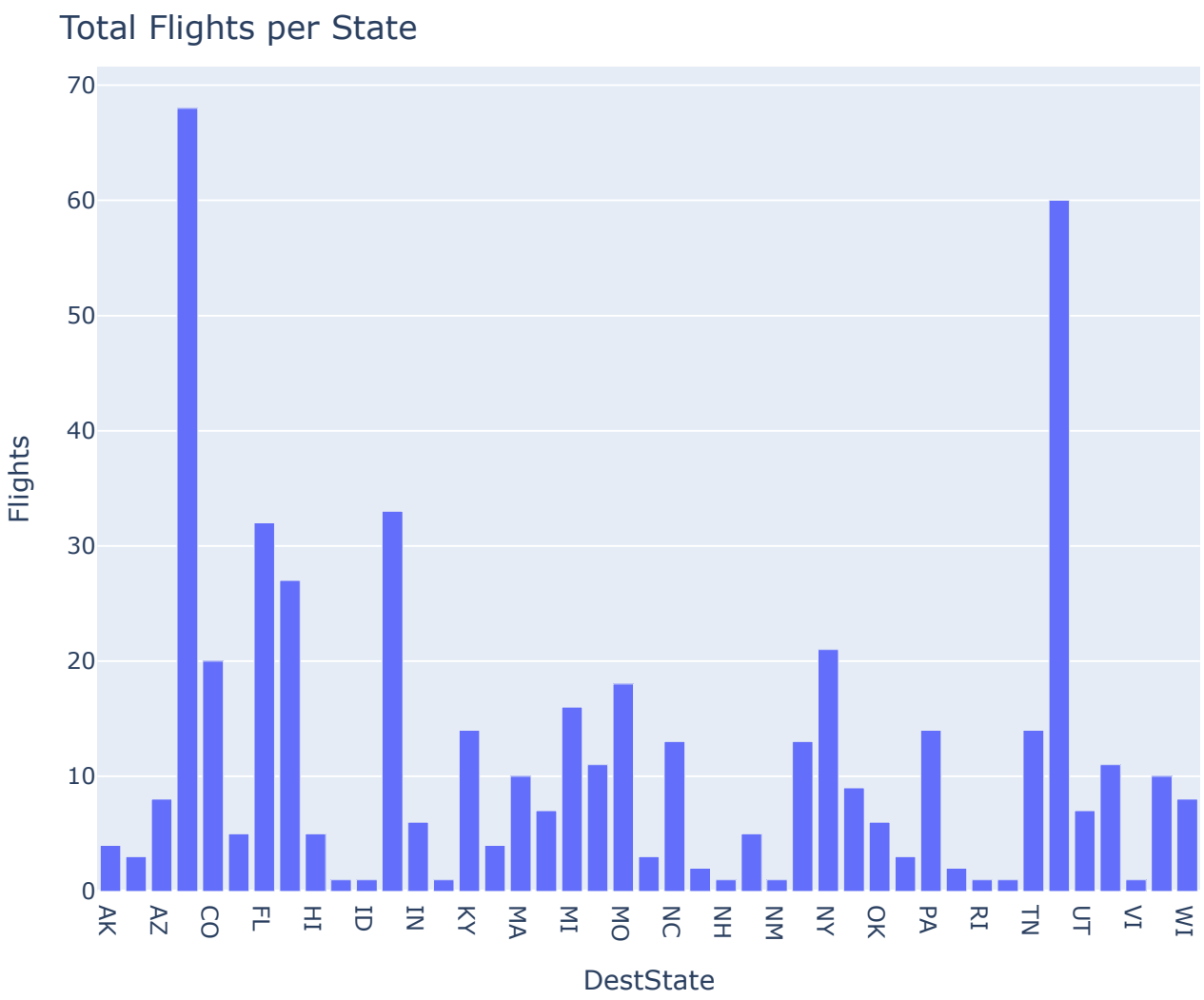
I want to compare how many flights go to each destination state.

```
bar_data = data.groupby(['DestState'])['Flights'].sum().reset_index()
bar_data
```

	DestState	Flights
0	AK	4.0
1	AL	3.0
2	AZ	8.0
3	CA	68.0
4	CO	20.0

`Result:` A table listing each destination state code alongside its total flight count.

```
fig = px.bar(bar_data, x="DestState", y="Flights", title='Total Flights per
State')
fig.show()
```

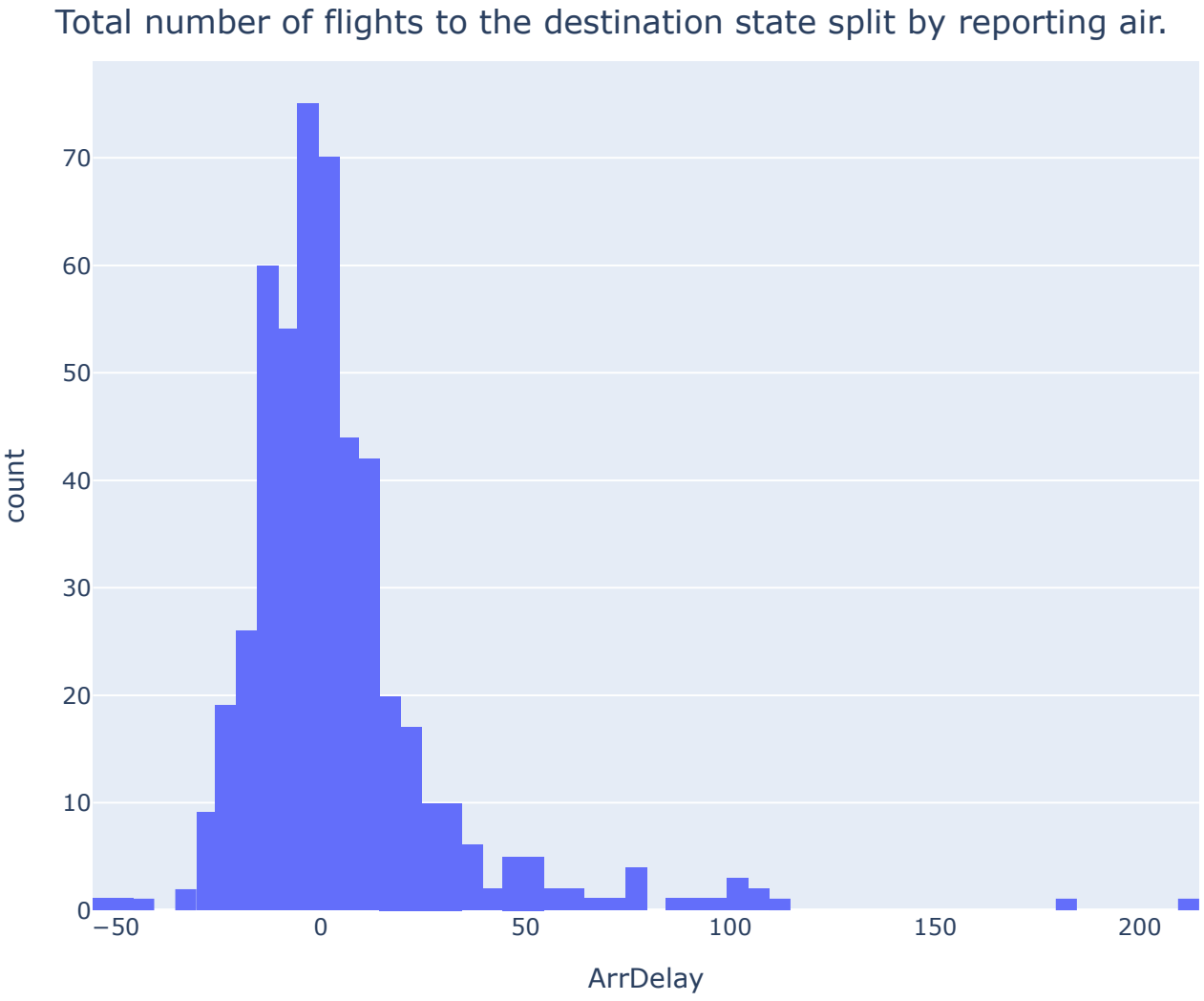


`Result:` California (CA) receives the most flights - about 68 - while Vermont (VT) has only 1 flight in the sample.

4.4. **Histogram**- Arrival Delay Distribution

I want to understand how arrival delays are spread across flights. I first **fill any missing delay values with 0** so the histogram reflects all rows.

```
data['ArrDelay'] = data['ArrDelay'].fillna(0)
fig = px.histogram(data, x="ArrDelay", title="Total number of flights to the
destination state split by reporting air.")
fig.show()
```



Result: Positive values, the flight arrived late.Negative values, the flight arrived early (ahead of schedule), most delays are short.

4.5. **Bubble Plot** - Flights per Reporting Airline

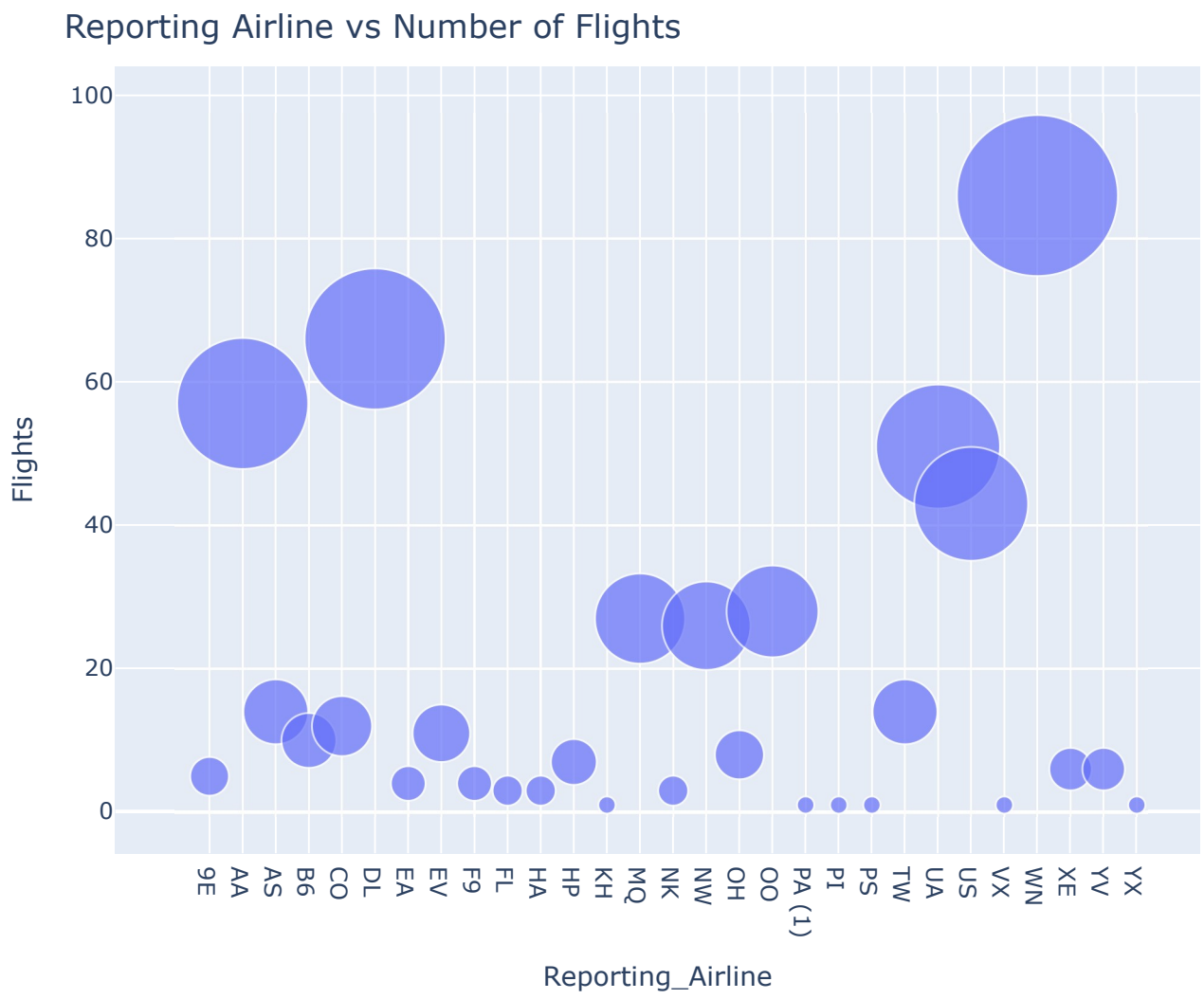
I want to compare total flight volumes across airlines at a glance.

```
bub_data = data.groupby('Reporting_Airline')['Flights'].sum().reset_index()
bub_data
```

	Reporting_Airline	Flights
0	9E	5.0
1	AA	57.0
2	AS	14.0
3	B6	10.0
4	CO	12.0

Result: A dataframe with each reporting airline code and its summed flight count.

```
fig = px.scatter(bub_data, x="Reporting_Airline", y="Flights", size="Flights",
                 hover_name="Reporting_Airline", title='Reporting Airline vs
Number of Flights', size_max=60)
fig.show()
```

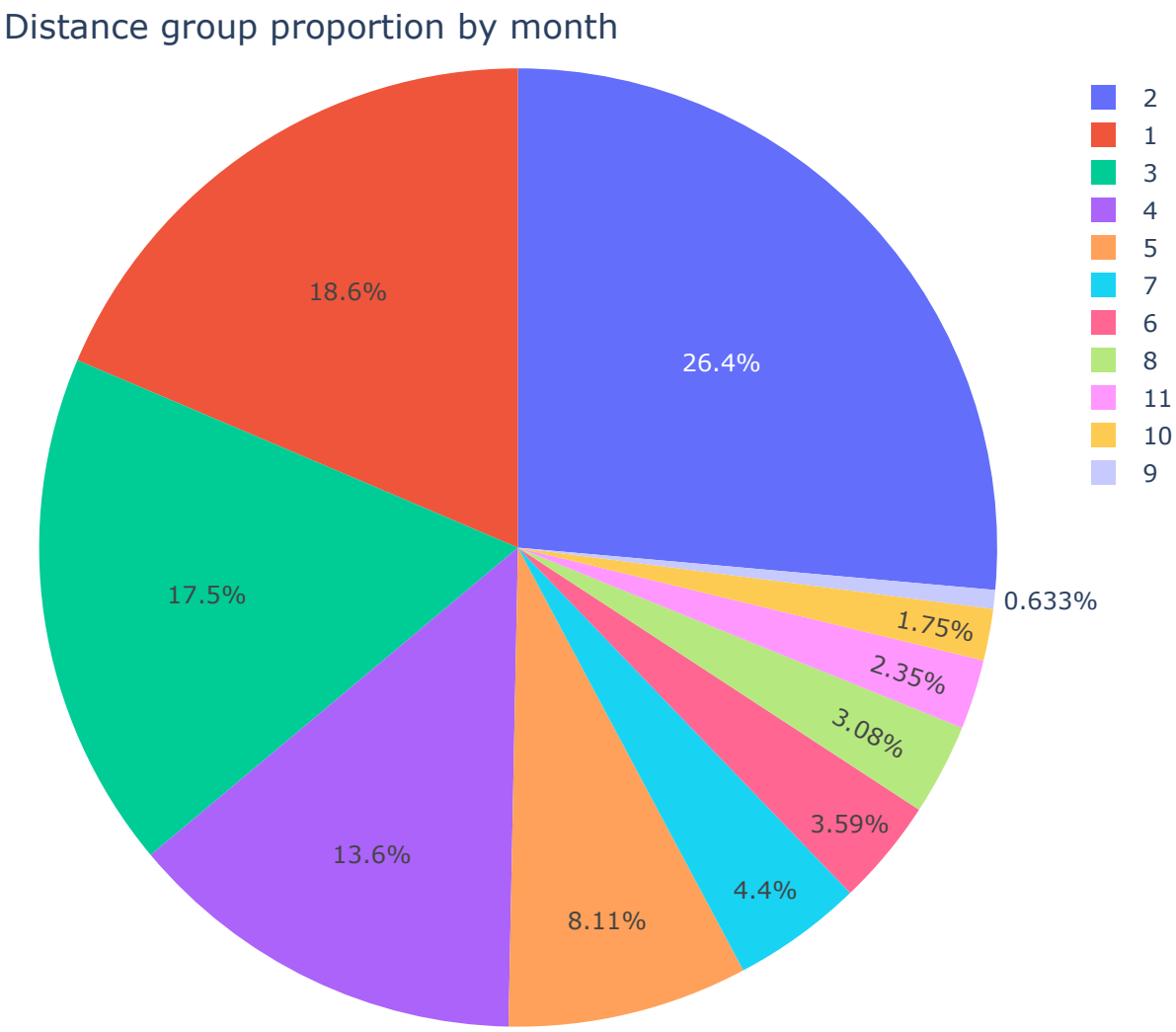


Result: Airline WN (Southwest) dominates with roughly 86 flights - the largest bubble by a clear margin.

4.6. **Pie Chart** - Distance Group Proportion by Month

I want to see how distance-group categories are proportionally distributed across months.

```
fig = px.pie(data, values='Month', names='DistanceGroup', title='Distance group proportion by month')
fig.show()
```



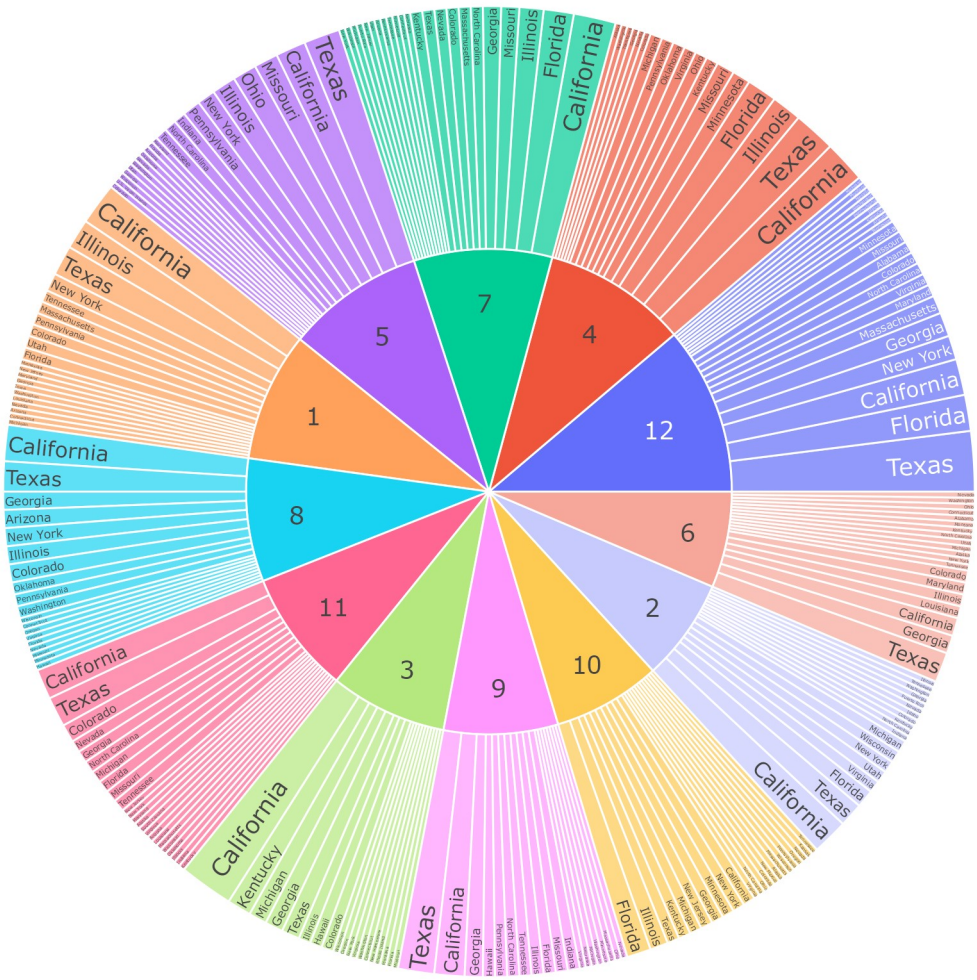
Result: February accounts for the largest proportion among the distance groups shown.

4.7. Sunburst Chart - Flight Distribution Hierarchy

I want a hierarchical breakdown of flight counts by month, then further by destination state.

```
fig = px.sunburst(data, path=['Month', 'DestStateName'], values='Flights', title='Flight Distribution Hierarchy')
fig.show()
```

Flight Distribution Hierarchy



`Result:` The innermost ring shows months as the root level; the outer ring breaks each month into destination states. Sector sizes reflect flight counts, making it easy to spot which month–state combinations carry the heaviest traffic.

That's the full walkthrough. This lab gave me hands-on practice building seven chart types with both Plotly Graph Objects and Plotly Express, and applying them to real airline data to uncover patterns in delays, distances, and flight distribution.